A project report on

GRAPH BASED PREDICTIVE AND PRESCRIPTIVE ANALYTICS FOR CROSS BORDER POWER GRID NETWORKS

Submitted as part of the Predictive Analytics course

Bachelor of Technology in Computer Science and Engineering

by

Prakki Krishna Karthik (23BCE1294)
Vasudev (23BCE1496)
Shubham Bhuyan (23BCE1809)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
Faculty: Pattabiraman V
October 2026

ACKNOWLEDGEMENT

We express our sincere gratitude to Pattabiraman V, faculty for the Predictive Analytics course, for the guidance and evaluation framework that shaped this project. The course topics on preprocessing, model comparison, time-series forecasting, classification metrics, class imbalance, and decision support were used to design the complete workflow.

We thank the project data providers represented by the supplied dynamic_with_flows.csv and power_flow.csv files. These files allowed us to construct a country-interconnector graph, replay historical hourly observations, compare forecasting models, and study the relationship between prediction and recommended action.

We also acknowledge the research community whose work on gated recurrent units, diffusion convolutional recurrent neural networks, graph time-series forecasting, and power-system graph learning informed the design. The report distinguishes ideas adopted from prior work from the applied integration implemented in GridPulse.

Finally, we acknowledge the importance of stating limitations clearly. The project is a research and course prototype. It does not replace a power-flow solver, protection study, N-1 security analysis, or the decisions of a qualified grid operator.

ABSTRACT

Cross-border power grids are naturally represented as networks: countries or grid areas form nodes, while interconnectors form edges over which power moves. This project develops GridPulse, an explainable predictive and prescriptive analytics prototype that uses two supplied datasets to forecast next-hour country net flow, classify stress, estimate a transparent distance-based loss proxy, and test whether a stressed connection has a lower-risk alternate route.

The pipeline processes 315,598 dynamic country records and 1,262,592 directional power-flow records covering 12 countries and 24 canonical links from 2022 to 2024. It validates missing values, duplicates, aliases, and timestamp gaps; creates a common hourly history; builds local, global, and graph features; and evaluates models using chronological holdout data. The saved comparison reports Persistence RMSE of 580.3 MW, Fusion GRU RMSE of 570.5 MW, and DCRNN RMSE of 634.0 MW. Fusion GRU is the research winner by RMSE, while the operational dashboard keeps Persistence as the selected baseline because its production artifact uses the lowest pooled chronological RMSE and is simpler to audit.

The prescriptive module does not switch the grid. It estimates moved megawatts, searches alternate paths, rejects links with insufficient proxy headroom or Critical screening state, ranks feasible candidates by distance, utilization, risk, and proxy loss, and reports best-effort bottlenecks when a fully feasible path does not exist. The main contribution is an auditable integration of data quality, graph features, predictive evaluation, distance-loss assumptions, and recommendation guardrails rather than an unsupported claim of a new neural architecture.

KEYWORDS

Predictive analytics; prescriptive analytics; cross-border power flow; graph time series; Fusion GRU; DCRNN; rerouting; physical-loss proxy; explainable decision support; model evaluation.

CONTENTS

KEYWORDS 3
LIST OF FIGURES 7
LIST OF TABLES 7
ABBREVIATIONS 7
CHAPTER 1 INTRODUCTION 8
1.1 Background 8
1.2 Motivation 8
1.3 Objectives 9
1.4 Scope 9
1.5 Research questions 9
CHAPTER 2 DOMAIN AND PROBLEM FORMULATION 10
2.1 Cross-border power-flow domain 10
2.2 Predictive formulation 10
2.3 Prescriptive formulation 10
2.4 User and system requirements 10
2.5 Assumptions 10
CHAPTER 3 LITERATURE REVIEW 11
3.1 Recurrent neural forecasting 11
3.2 Diffusion convolutional recurrent networks 11
3.3 Adaptive graph forecasting 11
3.4 Power-system graph learning 11
3.5 Comparison of existing approaches 11
3.6 Identified gap 11
CHAPTER 4 DATASET AND PREPROCESSING 12
4.1 Data sources 12
4.2 Dataset schema 12
4.3 Preprocessing workflow 12
4.4 Data quality results 12
4.5 Samples and labels 13
4.6 Leakage controls 13
CHAPTER 5 FEATURE ENGINEERING AND GRAPH CONSTRUCTION 14
5.1 Graph construction 14
5.2 Feature groups 14
5.3 Feature catalog 14
5.4 Distance and physical loss proxy 14
CHAPTER 6 PREDICTIVE METHODOLOGY 16

6.1 End-to-end analytical workflow 16
6.2 Model families 16
6.3 Training protocol 16
6.4 Classical forecasting models 16
Persistence 16
Twenty four hour moving average 16
Seasonal naive 17
Clamped linear trend 17
Why baselines matter 17
6.5 Fusion GRU 17
6.6 Diffusion Convolutional Recurrent Neural Network 17
6.7 Why DCRNN can lag 17
6.8 Evaluation metrics 18
CHAPTER 7 SYSTEM ARCHITECTURE AND TECHNOLOGY STACK 19
7.1 Architecture 19
7.2 Technology stack 19
7.3 Backend and frontend integration 19
7.4 Live replay behavior 19
7.5 Security and validation posture 20
CHAPTER 8 IMPLEMENTATION AND EXPERIMENTAL DESIGN 21
8.1 Pipeline implementation 21
8.2 Experiment implementation 21
8.3 Reproducibility 21
8.4 Frontend implementation 21
8.5 Explainability principles 21
8.6 Testing 21
CHAPTER 9 RESULTS AND EVALUATION 22
9.1 Model comparison 22
9.2 Training and validation curves 22
9.3 ROC AUC and precision recall analysis 23
9.4 Feature interpretation and error analysis 24
9.5 Learned-state projection and confusion matrix 25
DCRNN confusion matrix 25
9.6 Metric interpretation 25
9.7 Results summary 26
CHAPTER 10 PRESCRIPTIVE ANALYTICS AND REROUTING 27
10.1 Decision sequence 27
10.2 Why this is prescriptive 27

CHAPTER 11 DISCUSSION NOVELTY LIMITATIONS AND ETHICS 28
11.1 What is new in this project 28
11.2 Technical limitations 28
11.3 Ethical and operational safeguards 28
CHAPTER 12 CONCLUSION AND FUTURE WORK 29
12.1 Conclusion 29
12.2 Team contribution 29
12.3 Future work 29
REFERENCES 30
APPENDIX A COMPLETE FEATURE INVENTORY 30
APPENDIX B PROJECT ARTIFACT MAP 30
APPENDIX C DCRNN TRAINING OUTPUT 32
APPENDIX D DATASET SCHEMA AND PROVENANCE 33
Provenance rules 33
APPENDIX E COMPLETE FEATURE ENGINEERING DETAILS 34
APPENDIX F DCRNN COMPUTATION WALKTHROUGH 35
APPENDIX G BACKEND API AND LIVE REPLAY DATA FLOW 36
Snapshot calculation 36
Interface honesty 36
APPENDIX H WORKED REROUTING CALCULATION 37
Path ranking 37
APPENDIX I EVALUATION METRIC INTERPRETATION 38
Why thresholds are based on the training data 38
APPENDIX J TESTING AND VALIDATION MATRIX 39
Validation boundaries 39
APPENDIX K REPRODUCIBILITY AND ARTIFACT MAP 40
Saved evidence 40

LISTS AND ABBREVIATIONS

LIST OF FIGURES

- Figure 1 Project end-to-end workflow
- Figure 2 Original model comparison figure
- Figure 3 Fusion GRU training curves
- Figure 4 DCRNN training curves
- Figure 5 ROC and precision-recall curves
- Figure 6 DCRNN ROC and precision-recall curves
- Figure 7 Feature importance and ablation evidence
- Figure 8 Fusion GRU test-error distribution
- Figure 9 t-SNE projection of learned states

LIST OF TABLES

- Table 1 Dataset profile
- Table 2 Preprocessing and split configuration
- Table 3 Engineered feature catalog
- Table 4 Forecasting model comparison
- Table 5 Confusion matrix values
- Table 6 Technology stack and uses

ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
DCRNN	Diffusion Convolutional Recurrent Neural Network
F1	Harmonic mean of precision and recall
GRU	Gated Recurrent Unit
MAE	Mean absolute error
MAPE	Mean absolute percentage error
RMSE	Root mean square error
SSE	Server-sent events
t-SNE	t-distributed stochastic neighbor embedding

CHAPTER 1 INTRODUCTION

1.1 Background

Modern electricity systems are not isolated time series. Power is generated, consumed, and exchanged over interconnected networks. A change in one country can alter flows in neighboring countries, and a stressed transfer can make alternate paths important. This motivates a graph-based view in which the node state contains recent load and net flow while the edge state contains interconnector direction, magnitude, utilization, and distance.

Predictive analytics asks what may happen in the next period. Prescriptive analytics follows with a constrained question: if the predicted condition is undesirable, what action should an operator investigate? These questions are related but not identical. A model can forecast a high flow without proving that a switch or dispatch action is safe. GridPulse therefore separates prediction from recommendation and places engineering validation checks beside every route result.

1.2 Motivation

Graph neural-network research shows why topology can matter for spatio-temporal forecasting. DCRNN models diffusion on a directed graph and combines it with recurrent temporal dynamics [4]. Graph WaveNet shows an alternative in which hidden spatial dependencies are learned alongside temporal convolutions [5]. Power-system surveys identify forecasting, state estimation, fault diagnosis, and topology management as important graph-learning applications while also emphasizing robustness and domain knowledge [7, 8].

Project motivation. Create a transparent course-scale system that shows how the data, graph, model, evaluation evidence, physical assumptions, and recommended route connect end to end.

1.3 Objectives

1. Build a reproducible preprocessing pipeline for the supplied country and interconnector CSV files.
2. Represent the cross-border network as canonical nodes and edges while preserving alias information.
3. Create time-series and graph features for next-hour net-flow forecasting.
4. Compare transparent classical baselines with Fusion GRU and DCRNN under the same chronological protocol.
5. Evaluate both continuous prediction error and Normal, Warning, and Critical stress classes.
6. Calculate a clearly labeled distance-based resistance and I^2R loss proxy using constant assumptions.
7. Provide a recommendation-only rerouting studio with headroom and risk guardrails.
8. Expose results through a browser dashboard with live historical replay and explainable values.

1.4 Scope

The scope includes historical replay, model training artifacts, a lightweight Python backend, a browser frontend, evaluation figures, and what-if route search. The scope excludes automatic generation dispatch, automatic switching, protection coordination, guaranteed thermal limits, real-time utility telemetry, market optimization, and engineering-grade AC power-flow validation.

1.5 Research questions

- Does a trained temporal model improve on a persistence baseline for next-hour net flow?
- Does preserving the country graph through diffusion improve forecasting on this dataset?
- Which features most influence the saved validation error?
- Can a stressed link be mapped to a candidate route without hiding infeasible or unsafe conditions?

CHAPTER 2 DOMAIN AND PROBLEM FORMULATION

2.1 Cross-border power-flow domain

The project treats each country code as a node. Each power-flow record contains a timestamp, interconnector label, source, target, and power flow in megawatts. Multiple directional records can refer to one physical connection. The pipeline converts them into a sorted canonical edge such as DE-FR while retaining the raw alias mapping for audit. At each time point, forward and reverse values are combined into a signed net value and an observed magnitude used for screening.

2.2 Predictive formulation

For a country i and time t, let x_i,t contain recent net flow, load, imbalance, load residual, shared network context, and static graph properties. The model receives a 24-hour window and predicts the next-hour net flow $\hat{y}_i,t+1$. The output is continuous in megawatts and is also converted to a screening class by comparing absolute flow magnitude with training-derived warning and critical thresholds.

$$\hat{y}_{i,t+1} = f_\theta(x_{i,t-23}, \ldots, x_{i,t}; G)$$

The graph G is used explicitly by DCRNN and indirectly through engineered centrality features in Fusion GRU.

2.3 Prescriptive formulation

If the selected edge e is stressed, the what-if module chooses a reduction percentage p and computes the amount of flow to investigate. It then removes the selected edge from the visible graph, searches connected paths between the same endpoints, applies hard proxy guardrails, and ranks surviving paths. The output is a recommendation with status, route nodes, route edges, bottlenecks, loss comparison, reason, and required engineering checks.

2.4 User and system requirements

Requirement group	Implemented behavior
Graph representation	Country nodes, canonical interconnector edges, direction, magnitude, coordinates, and risk colors
Data validation	Missing cells, duplicates, country codes, timestamp gaps, aliases, and engineering assumptions are recorded
Prediction	Classical baselines plus Fusion GRU and DCRNN research artifacts
Risk classification	Normal, Warning, and Critical states from training quantiles and screening utilization
Prescriptive analysis	Alternate-path search, headroom checks, critical-link rejection, cost ranking, and best-effort bottlenecks
Interface	Animated topology, replay stream, forecast, metrics, physical-loss lens, and reroute studio

2.5 Assumptions

- All timestamps are parsed as UTC and aligned on an hourly basis.
- Country centroids approximate interconnector distance when surveyed line geometry is absent.
- Observed 95th percentile flow magnitude acts as a capacity proxy for screening only.
- Warning and Critical thresholds are the 75th and 90th training quantiles of absolute net flow.
- The stream is a historical replay that demonstrates the interface without claiming live telemetry.

CHAPTER 3 LITERATURE REVIEW

3.1 Recurrent neural forecasting

The gated recurrent unit was introduced as part of an RNN encoder-decoder framework that learns sequence representations through update and reset gates [6]. A GRU is useful for this project because hourly load and flow exhibit temporal dependence, while the gated state can retain recent information without requiring a large architecture. GridPulse uses a compact Fusion GRU over engineered local, global, and static features.

3.2 Diffusion convolutional recurrent networks

DCRNN was proposed for traffic forecasting as a model that represents spatial dependence as diffusion on a directed graph and temporal dependence through a recurrent architecture [4]. The central idea transfers naturally to the project graph: information from neighboring countries can influence the next-hour estimate. The implementation uses forward and reverse row-normalized transition matrices and two diffusion steps before each recurrent update.

3.3 Adaptive graph forecasting

Graph WaveNet argues that a known topology may not capture every dependency and proposes an adaptive dependency matrix together with dilated temporal convolutions [5]. This is relevant as future work. The current project deliberately keeps the topology inspectable and fixed so that the route planner, graph visualization, and physical assumptions remain understandable to a non-specialist audience.

3.4 Power-system graph learning

Recent reviews describe applications of graph computing and GNNs in power-system forecasting, state estimation, fault diagnosis, topology identification, and decision support [7, 8]. These reviews support using a graph representation but also motivate transparent validation, robustness analysis, and incorporation of engineering constraints rather than reporting accuracy alone.

3.5 Comparison of existing approaches

Approach	Strength	Limitation for this project
Classical persistence	Very fast, strong for autocorrelated short-horizon flow, easy to audit	Does not use graph or exogenous context
Moving average and seasonal naive	Simple reference rules for time-series behavior	Smooth or lag abrupt changes and may fail near sign changes
Fusion GRU	Learns nonlinear temporal relationships from engineered local and global inputs	Does not directly diffuse hidden state over graph edges
DCRNN	Combines directed graph diffusion with recurrent temporal memory	More compute and may not improve when local history dominates
Graph WaveNet style models	Can learn hidden dependencies and long temporal receptive fields	More complex; requires stronger audit and ablation evidence

3.6 Identified gap

Many forecasting papers optimize prediction error on curated benchmarks. A deployable-looking course prototype must also answer less glamorous questions: were directional aliases normalized, are the timestamps complete, which outputs are measured versus estimated, what happens when physical line parameters are missing, and does a complex graph model actually beat a simple baseline on this dataset? GridPulse addresses this gap by making data quality, assumptions, model selection, failure cases, and route guardrails visible.

Novelty position. The defensible novelty is the auditable integration of data quality, graph features, evidence-driven model selection, proxy physics, and stress-aware counterfactual routing. It is not a claim of a new neural layer.

CHAPTER 4 DATASET AND PREPROCESSING

4.1 Data sources

The project uses two supplied CSV files. The dynamic file contains country-level hourly variables such as actual load, day-ahead load forecast, net flow, flow imbalance, renewable generation, weather variables, socio-economic variables, and graph centralities. The power-flow file contains timestamped directional interconnector records. The two datasets are joined by time and network identifiers after validation and canonicalization.

Dataset property	Value
Dynamic records	315,598
Dynamic columns	37
Power-flow records	1,262,592
Power-flow columns	5
Countries or nodes	12
Canonical links or edges	24
Time range	2022-01-01T00:00:00Z to 2024-12-31T22:00:00Z
Retained history per country	960 points

4.2 Dataset schema

The principal dynamic fields are datetime, country, Actual_Total_Load_MW, Day_ahead_Total_Load_Forecast_MW, load_residual, net_flow, flow_imbalance_ratio, renewable generation fields, calendar encodings, weather fields, location, socio-economic fields, and graph centralities. The power-flow schema is datetime, interconnector, source, target, and power_flow_mw. The complete column inventory is retained in results/dataset_profile.json and is summarized in Appendix A.

4.3 Preprocessing workflow

Step	Operation	Purpose
1	Parse	Read UTF-8 BOM-safe CSV files and parse UTC timestamps
2	Validate	Count missing cells, duplicates, invalid codes, and hourly gaps
3	Canonicalize	Map raw interconnector labels to sorted endpoint IDs while retaining aliases
4	Align	Retain common recent history and align country features by hourly timestamp
5	Scale	Fit means and standard deviations on training data only
6	Split	Hold out the final 168 hours chronologically without random leakage
7	Evaluate	Compare baselines, Fusion GRU, and DCRNN on the same target and horizon

4.4 Data quality results

- Dynamic missing cells: 0; power-flow missing cells: 0.
- Dynamic duplicate keys: 0; power-flow duplicate keys: 0.
- Country timestamp gaps: 38 hours across several countries.
- Interconnector aliases normalized: 7.

Why chronological splitting matters. Randomly mixing future and past rows would allow future operating patterns to influence training. The project uses a final 168-hour chronological holdout so the evaluation resembles forecasting unseen future hours.

4.5 Samples and labels

Configuration	Value
Sequence length	24 hours
Window stride	4 hours
Test horizon	168 hours
Fusion GRU train samples	1,800
Fusion GRU validation samples	504
Fusion GRU test samples	2,016
Warning threshold	3550.44 MW
Critical threshold	5941.40 MW

The label is not an externally certified protection state. It is a three-class screening label derived from the absolute net-flow magnitude. Values below the 75th training quantile are Normal, values between the 75th and 90th quantiles are Warning, and values at or above the 90th quantile are Critical. This makes the class metrics useful for model comparison while preserving the distinction between analytics and engineering certification.

4.6 Leakage controls

- Feature and target scalers are fit on the training window only.
- Validation and test values are transformed using training statistics.
- The final 168 hours are never used to construct training windows.
- The same chronological test horizon is used for the trained model comparison.

CHAPTER 5 FEATURE ENGINEERING AND GRAPH CONSTRUCTION

5.1 Graph construction

The graph G = (V, E) contains 12 country nodes and 24 canonical interconnector edges. Country centroids from the dynamic file supply latitude and longitude. Each raw direction is stored for audit, while the canonical edge is built by sorting its endpoints. This prevents the system from treating DE-FR and FR-DE as unrelated physical assets.

5.2 Feature groups

Feature group	Features	Role
Local time series	local net flow, local load, local imbalance, local load residual	Recent behavior and country-specific operating pressure
Network context	global net flow, global load, global imbalance, global residual	Shared system conditions across countries
Static graph	degree, closeness, betweenness centrality	Connectivity and bridge-node context

5.3 Feature catalog

Feature	Construction	Why it is used
local net flow	observed country net flow at t	The target's recent history gives the model an autoregressive anchor.
local load	observed country load at t	Demand pressure changes the likely transfer requirement.
local imbalance	inflow/outflow imbalance signal	Large imbalance separates stable transfer from directional stress.
local load residual	actual load - day-ahead forecast	Forecast misses can precede unexpected flow changes.
global net flow	mean(net flow across countries)	A node's forecast is conditioned on the system-wide state.
global load	mean(load across countries)	Shared demand conditions propagate through interconnectors.
global imbalance	mean(abs(imbalance) across countries)	System-wide directional pressure provides context around a node.
global residual	mean(abs(residual) across countries)	Collective forecast error can signal an unstable operating period.
degree centrality	degree(node)/(N-1)	A node with more neighbors has more pathways for pressure to move.
closeness centrality	inverse mean shortest-path distance	A node's graph proximity gives a compact connectivity context.
betweenness centrality	fraction of shortest paths through node	Bridge nodes can matter to rerouting decisions.

5.4 Distance and physical loss proxy

The datasets do not provide surveyed line corridors, conductor material, conductor cross-section, actual voltage, power factor, or thermal capacity. The project therefore calculates a transparent scenario proxy. Haversine distance is applied between country centroids. Resistance is derived from assumed resistivity, length, and area. Apparent power and current are then estimated from observed active power, power factor, and line voltage.

$$a = \sin^2(\Delta\phi/2) + \cos(\phi_1)\cos(\phi_2)\sin^2(\Delta\lambda/2)$$

$$d = 2R_e \arcsin\left(\sqrt{a}\right)$$

Haversine distance in kilometres after unit conversion.

$$R = \frac{\rho L}{A}$$

Resistance in ohms, where ρ is resistivity, L is length, and A is conductor area.

$$S = \frac{|P|}{pf}, \quad I = \frac{S}{\sqrt{3}V}$$

$$P_{\text{loss}} = 3I^2R$$

Simplified three-phase resistive loss estimate.

Assumption	Value
Line voltage	400 kV
Power factor	0.95
Copper resistivity	1.68×10^{-8} Ω m
Cross-sectional area	2,000 mm²
Length	Haversine centroid-to-centroid distance

Interpretation. This output is a distance-based flow-loss estimate, not a substitute for an AC power-flow solver. The report keeps the assumptions visible so they can be replaced when engineering data becomes available.

CHAPTER 6 PREDICTIVE METHODOLOGY

6.1 End-to-end analytical workflow

Stage	Input	Output
Ingest	Two CSV files	Parsed dynamic and directional rows
Validate	Rows and keys	Quality counts, aliases, gaps, and limitations
Graph	Countries and interconnectors	12-node, 24-edge canonical graph
Features	History and graph metadata	12 × 960 × 11 feature tensor
Forecast	24-hour windows	Next-hour net-flow prediction
Classify	Actual and predicted flow	Normal, Warning, or Critical
Prescribe	Selected stressed edge	Route candidates and guardrails
Present	Saved artifacts and APIs	Animated browser dashboard

6.2 Model families

The study deliberately compares models of increasing complexity. Persistence provides an autoregressive baseline. Moving average and seasonal naive provide transparent alternatives. Linear trend tests whether a recent slope is enough. Fusion GRU learns temporal relationships from the engineered tensor. DCRNN adds directed graph diffusion before recurrent memory updates. This design prevents the project from assuming that the most complex model must win.

6.3 Training protocol

- Random seed: 42.
- Epochs: 40 for trained research models.
- Training-only standardization and chronological train, validation, and test windows.
- The same 24-hour lookback and 168-hour test horizon for Fusion GRU and DCRNN.
- Saved JSON, CSV, PNG, and checkpoint artifacts for reproducibility.

6.4 Classical forecasting models

Persistence

$$\hat{y}_{t+1} = y_t$$

The next hour is predicted as the latest observed flow.

Twenty four hour moving average

$$\hat{y}_{t+1} = \frac{1}{24} \sum_{i=0}^{23} y_{t-i}$$

Seasonal naive

$$\hat{y}_{t+1} = y_{t-24}$$

The same hour from the previous day is used when available.

Clamped linear trend

A least-squares line is fitted over the recent 24 observations. The extrapolated value is clamped to a margin around the observed window so a short data gap cannot create a runaway recommendation.

Why baselines matter

The selected dashboard baseline is Persistence because the production selection artifact chooses the lowest pooled chronological RMSE and because the rule is fast and easy to audit. Fusion GRU is still the research winner among the saved trained models by RMSE. Reporting both avoids conflating operational simplicity with research-model performance.

6.5 Fusion GRU

The Fusion GRU receives a 24-hour sequence for each country. At every hour the input combines four local signals, four global signals, and three static centrality values. The GRU updates a hidden state through gates, and a normalized linear head maps the final hidden state to the next-hour flow estimate. The model is called Fusion because local time-series information, global network context, and static graph information are concatenated before the recurrent layer.

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$$
$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Simplified GRU gate notation; the implementation uses PyTorch GRU cells.

The update gate controls how much new information is used. The reset gate controls how much previous state contributes to the candidate state. This lets the model preserve stable flow patterns while adapting when load, imbalance, or network context changes.

6.6 Diffusion Convolutional Recurrent Neural Network

DCRNN keeps the node axis intact. Instead of flattening the 12 countries into one vector, the model stores a hidden state for every country. Before the recurrent update, the graph convolution propagates each node's features through forward and reverse row-normalized transitions. Two diffusion steps allow information from immediate and second-order neighbors to influence the node representation.

$$X^{(0)} = X, \quad X^{(k)} = P X^{(k-1)}$$
$$H = [X^{(0)} \| X_f^{(1)} \| X_f^{(2)} \| X_r^{(1)} \| X_r^{(2)}] W + b$$

Forward and reverse diffusion features are concatenated before the recurrent gates.

In simple terms, Fusion GRU asks: what happened recently at this country and in the overall system? DCRNN asks the same question while also passing information along the known country-interconnector graph. The project uses DCRNN as a fair benchmark, not as a dispatch solver or a replacement for physical power-flow equations [4].

6.7 Why DCRNN can lag

- Recent local net flow is overwhelmingly the strongest saved feature, so additional diffusion can add noise rather than useful signal.
- The graph is a canonical country-interconnector topology, not a complete electrical admittance model.

- The dataset lacks validated line ratings, dispatch constraints, voltage states, and market shocks.
- DCRNN has higher inference time and more moving parts than Persistence or Fusion GRU.
- A weaker score is useful evidence: topology alone did not guarantee an advantage under this protocol.

6.8 Evaluation metrics

The report uses regression metrics for MW error and classification metrics for stress-state behavior. Because net flow can approach zero or change sign, MAPE is reported for course completeness but is not treated as the primary decision metric.

$$\mathrm{MAE} = \frac{1}{n}\sum |\hat{y}_i - y_i|$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum (\hat{y}_i - y_i)^2}$$

RMSE penalizes large errors more heavily than MAE.

$$\mathrm{MAPE} = \frac{100}{n}\sum \left|\frac{\hat{y}_i - y_i}{y_i}\right|$$

Unstable when actual values are near zero.

$$\mathrm{Accuracy} = \frac{TP + TN}{N}$$

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \quad \mathrm{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2\,\mathrm{Precision}\,\mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

The classification metrics are macro-averaged across Normal, Warning, and Critical classes. The confusion matrix is retained because a single accuracy value can hide whether Warning or Critical states are being confused with Normal. This follows the practical emphasis on class-aware evaluation in the course syllabus and the standard metric definitions used by machine-learning tooling [9].

CHAPTER 7 SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

7.1 Architecture

The project is organized into four layers. The data layer stores the supplied CSV files and compact processed artifacts. The analytical layer contains the standard-library preprocessing pipeline and the optional research experiment runner. The service layer exposes REST-style JSON endpoints and a server-sent event replay stream. The presentation layer uses HTML, CSS, and JavaScript to render the graph, animations, charts, metrics, and route what-if result.

The architecture intentionally keeps the production dashboard independent of the optional heavy experiment dependencies. This makes the dashboard runnable with the Python standard library while allowing NumPy, PyTorch, scikit-learn, and Matplotlib to be used for training and research artifact generation.

7.2 Technology stack

Technology	Use in the project
Python	Data ingestion, preprocessing, model evaluation, physics proxy, routing, and HTTP backend
NumPy	Tensor preparation, numerical calculations, and experiment arrays
PyTorch	Fusion GRU, DCRNN, training loop, checkpoints, and hidden-state extraction
scikit-learn	t-SNE projection and evaluation utilities
Matplotlib	Saved result figures for the report and presentation
HTML and CSS	Responsive dashboard structure and visual design
JavaScript	API polling, SSE replay, charts, network animation, and reroute interactions
JSON and CSV	Reproducible artifacts, API payloads, input data, and evaluation outputs

7.3 Backend and frontend integration

API route	Purpose
/api/health	Check service availability
/api/metadata	Return dataset, model, assumptions, and limitation metadata
/api/topology	Return nodes, edges, aliases, coordinates, and graph properties
/api/snapshot	Return a historical replay state at the selected time
/api/forecast	Return next-hour or multi-hour forecast values
/api/metrics and /api/validation	Return saved comparison and validation evidence
/api/physical-loss	Return distance, resistance, current, and proxy loss values
/api/route-plan and /api/what-if	Return alternate paths, bottlenecks, and loss comparison
/api/stream	Replay historical observations as a live-looking stream

7.4 Live replay behavior

The dashboard updates the visible snapshot, graph edge colors, flow particles, current metrics, and recommendation queue as replay time advances. The values change because the frontend requests different historical timestamps from the backend or consumes the replay stream. This is a real-time interface demonstration, not a live connection to a

transmission system. The interface surfaces Fusion GRU as the research model and labels Persistence as the lightweight operational baseline; this distinction prevents a presentation label from misrepresenting which endpoint is executing.

7.5 Security and validation posture

- What-if percentages are clamped before route calculation.
- Unknown edge identifiers are rejected.
- Route output is labeled recommendation-only.
- The interface exposes engineering checks instead of hiding missing parameters.

CHAPTER 8 IMPLEMENTATION AND EXPERIMENTAL DESIGN

8.1 Pipeline implementation

backend/pipeline.py performs two CSV passes. The first builds country histories and metadata. The second groups directional power-flow records by timestamp and canonical edge, computes forward, reverse, net, and magnitude values, and stores compact edge histories. It then derives the observed 95th percentile capacity proxy, distance, resistance, loss, utilization, and screening class.

8.2 Experiment implementation

backend/experiments.py loads processed artifacts, constructs the 12-node feature tensor, makes chronological samples, trains Fusion GRU and DCRNN, computes regression and classification metrics, produces ROC/PR curves, saves confusion matrices, runs permutation and ablation analysis, creates a t-SNE projection, and exports the light-background PNG figures used in this report.

8.3 Reproducibility

- Seed: 42.
- Configuration and feature names are saved in results/run_metadata.json.
- Every figure has a corresponding JSON or CSV source artifact.
- The saved model checkpoints are stored in results/models.
- Unit and smoke tests cover pipeline, routing, and API behavior.

Reproducibility rule. The report does not invent graph values in the document. The result figures and numerical tables are based on the saved artifacts produced by the project experiment layer.

8.4 Frontend implementation

The frontend is organized as a dashboard with overview, forecast, model evidence, physical-loss, and rerouting views. The graph view draws nodes and links, colors them by screening state, and animates moving particles to communicate direction. The forecast view plots recent history and future values. The model evidence view explains the saved metrics rather than treating a single accuracy value as sufficient. The physics view exposes the assumed voltage, power factor, resistivity, area, and derived distance. The reroute studio accepts a selected edge and reduction percentage, then displays the route status and bottlenecks returned by the backend.

8.5 Explainability principles

- Every major value is accompanied by a unit or interpretation.
- Measured, estimated, and proxy values are distinguished.
- A recommendation includes a reason and validation checks.
- The user can trace a route from selected edge to candidate path and bottleneck.
- Model comparison includes both simple baselines and neural models.

8.6 Testing

Test file	Coverage
tests/test_pipeline.py	Distance-loss calculation, thresholds, preprocessing and artifact behavior
tests/test routing.py	Path search, guardrails, no-change, feasible, and best-effort cases
tests/test_api.py	Endpoint smoke tests and response structure

CHAPTER 9 RESULTS AND EVALUATION

9.1 Model comparison

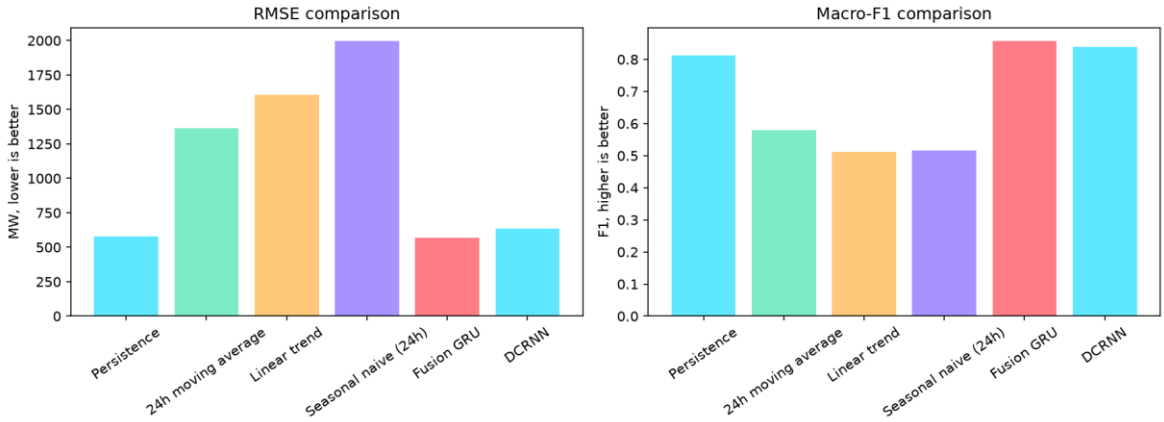


Figure 2 Original saved model comparison figure showing RMSE and macro F1.

Model	RMSE MW	MAE MW	Accuracy	Precision	Recall	F1	Time ms
Persistence	580.3	375.8	91.4%	81.3%	81.1%	81.2%	0.4
24h moving average	1363.4	947.1	81.3%	61.5%	55.5%	57.9%	12.8
Linear trend	1604.8	1156.9	74.4%	49.7%	54.1%	51.2%	78.0
Seasonal naive (24h)	1994.7	1310.5	75.4%	51.3%	51.9%	51.6%	0.6
Fusion GRU	570.5	400.3	92.7%	85.2%	86.2%	85.6%	34.7
DCRNN	634.0	436.2	92.5%	85.3%	82.7%	83.8%	63.6

Fusion GRU has the lowest saved RMSE among the trained research models at 570.5 MW and the highest F1 at 85.6%. Persistence has a slightly higher RMSE at 580.3 MW but is much faster and simpler. DCRNN has RMSE 634.0 MW and F1 83.8%, so graph diffusion did not improve this run. The dashboard selection artifact keeps Persistence as the operational baseline while the research comparison identifies Fusion GRU as the trained-model winner.

9.2 Training and validation curves

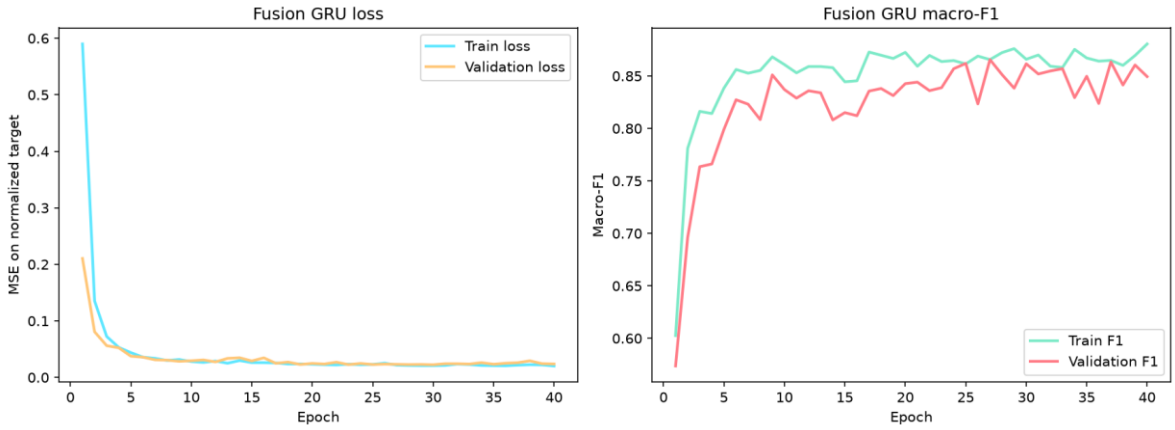


Figure 3 Original Fusion GRU training and validation loss and macro F1 curves.

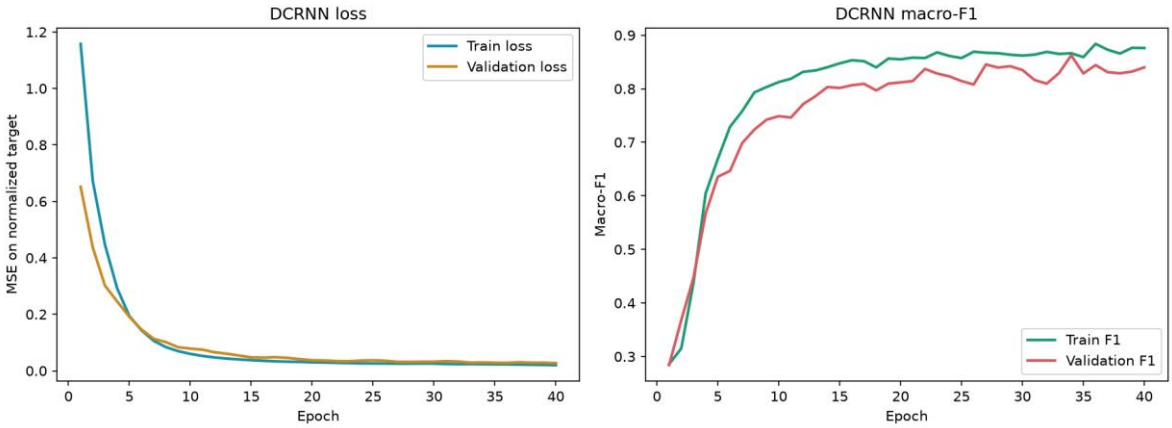


Figure 4 Original DCRNN training and validation loss and macro F1 curves.

The curves show that both trained models learn rapidly during the early epochs. Loss falls sharply, while macro F1 rises. The validation curves remain broadly aligned with training behavior, which is evidence against a severe training collapse in this run. The final test comparison is still more important than a visually good training curve because the holdout is chronological and represents unseen future-like hours.

Interpretation. A learning curve explains how the model learned; it does not prove that the model is engineering-safe. The saved test metrics and failure examples must be considered together.

9.3 ROC AUC and precision recall analysis

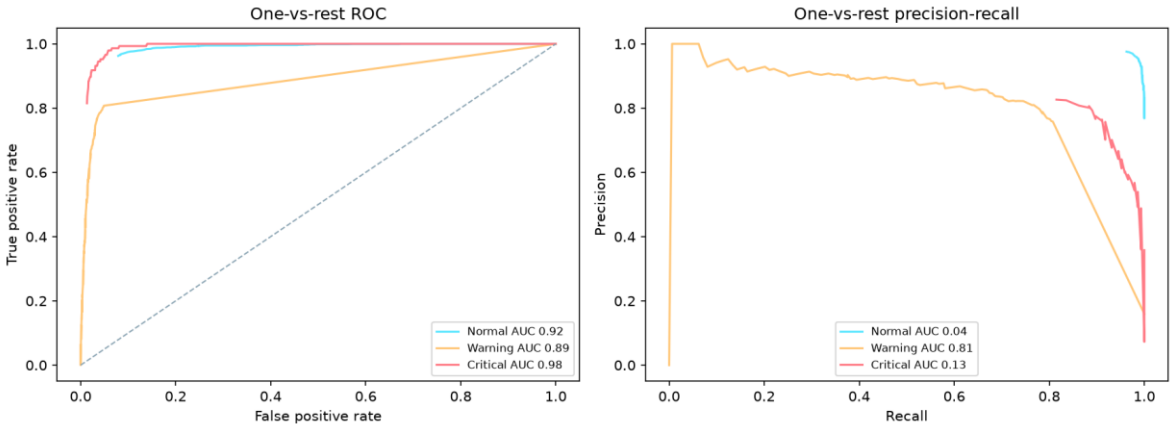


Figure 5 Original Fusion GRU ROC and precision-recall curves.

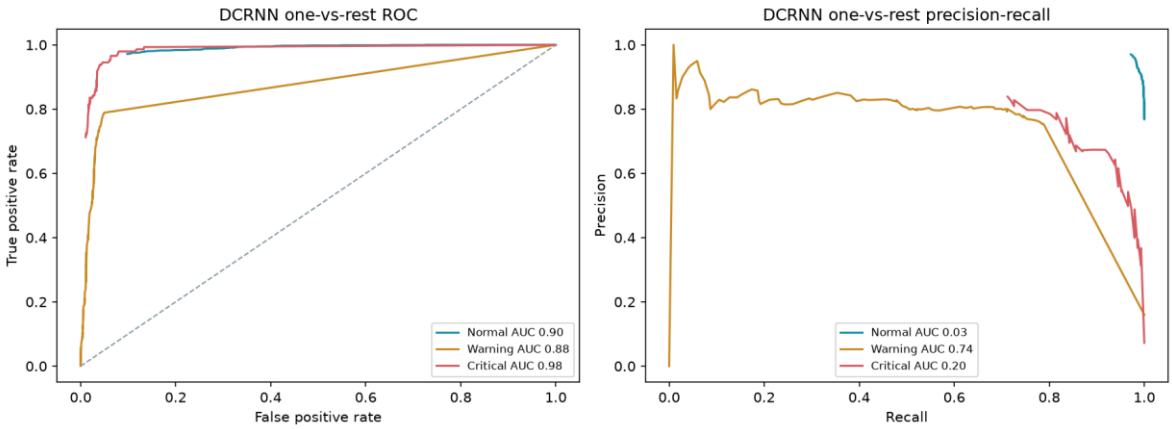


Figure 6 Original DCRNN ROC and precision-recall curves.

The ROC curves show the trade-off between true-positive rate and false-positive rate as the threshold changes. Precision-recall curves are particularly useful for Warning and Critical classes because those states are less frequent than Normal. A high area under a curve suggests good separability across thresholds, but the operating threshold still depends on the cost of missed stress versus false alarms. The dashboard therefore keeps the class thresholds explicit and includes a confusion matrix.

9.4 Feature interpretation and error analysis

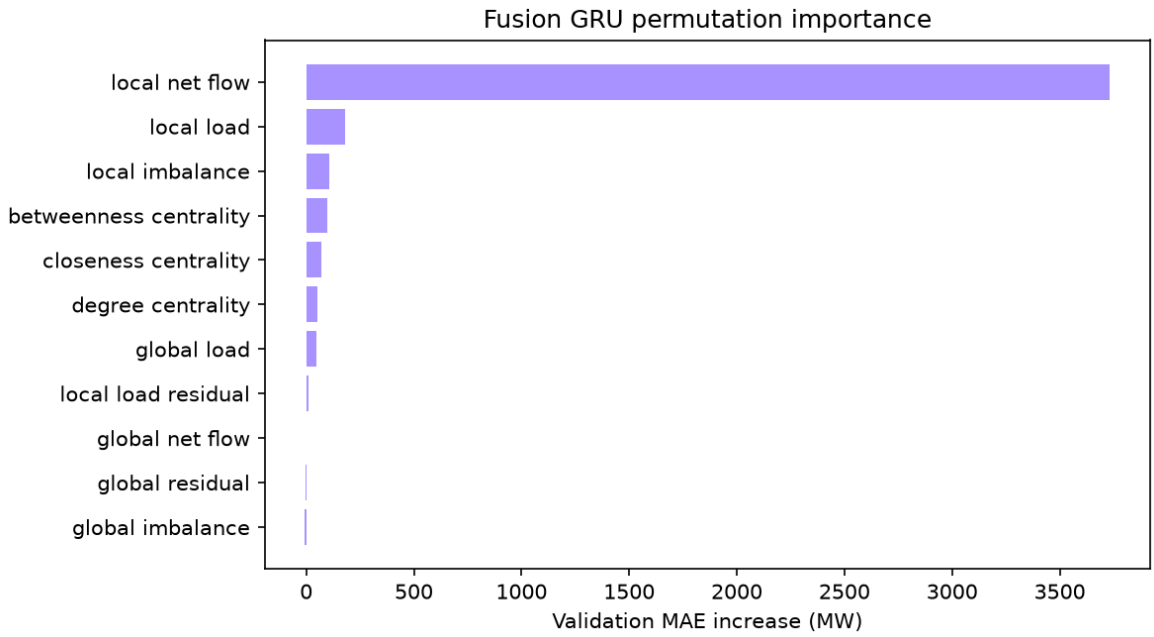


Figure 7 Original permutation-importance figure. Larger delta MAE means the model depends more on that feature.

The saved baseline validation MAE for the feature study is 374.9 MW. Shuffling local net flow increases MAE by 3733.1 MW, far more than any other feature. This supports the interpretation that recent local flow is the dominant predictive signal in this run. Centrality features contribute smaller but positive permutation deltas, while some global context features do not improve the validation MAE when shuffled. These results are model-specific evidence, not causal proof.

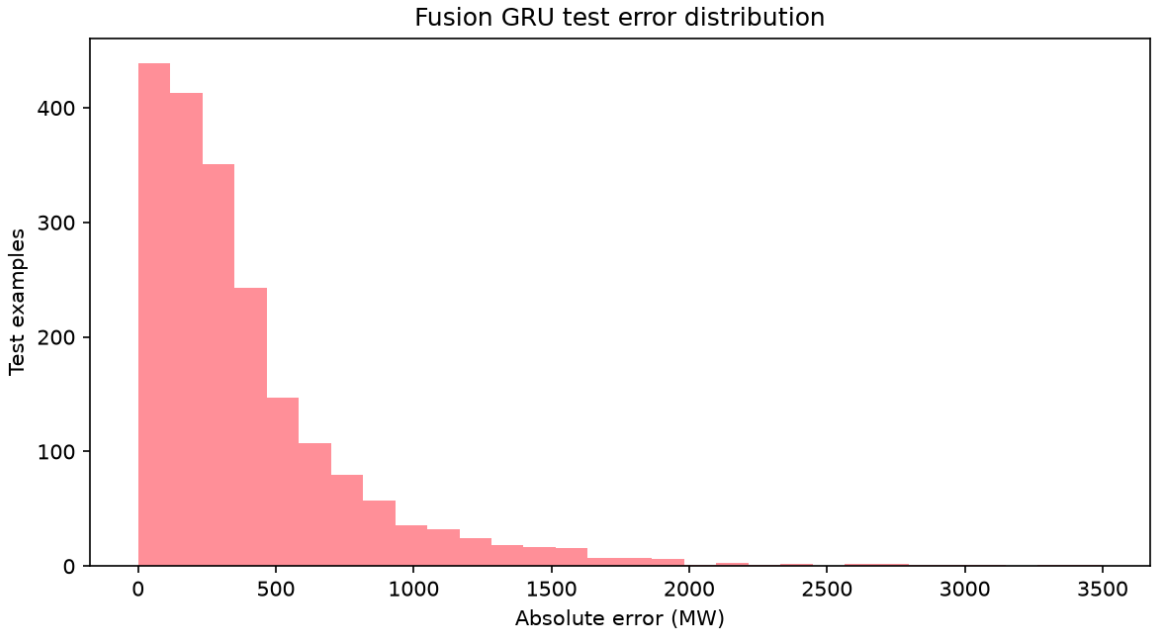


Figure 8 Original Fusion GRU test error distribution.

The error distribution is concentrated at smaller absolute errors with a long tail. The saved qualitative failures include a German Critical flow of 7,484.5 MW predicted as 3,986.7 MW, and a French Normal flow of -1,662.5 MW predicted as -5,030.8 MW. Sudden movements and class-boundary changes are therefore important failure cases.

9.5 Learned-state projection and confusion matrix

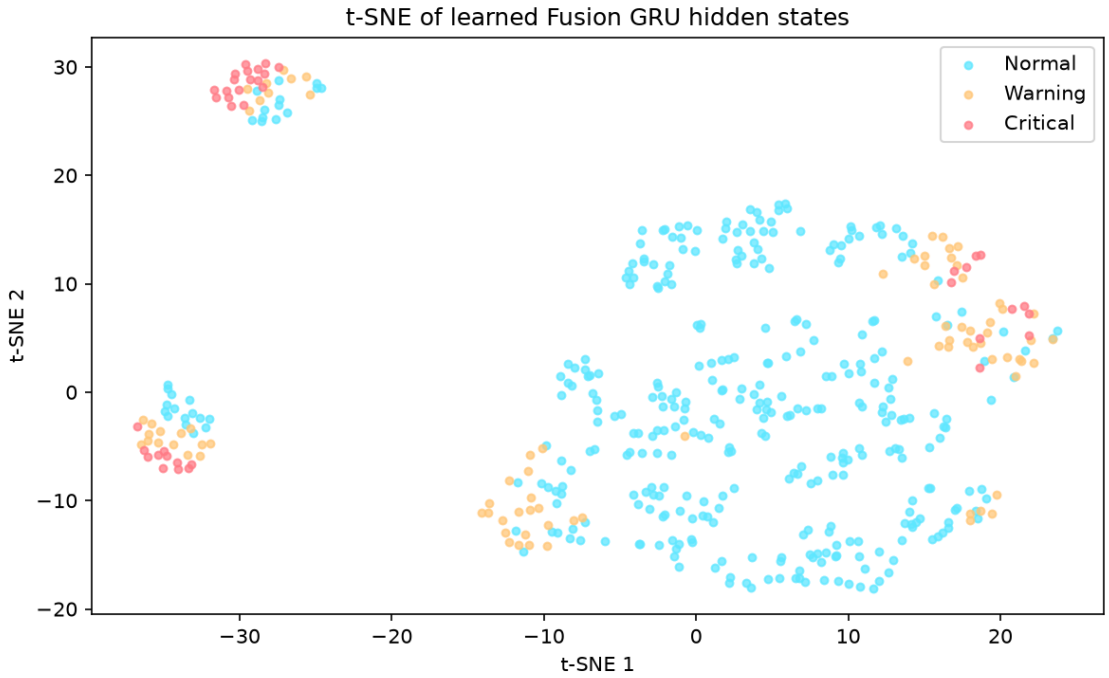


Figure 9 Original t-SNE projection of learned hidden states.

The t-SNE projection provides a two-dimensional view of hidden states and can show whether similar operating conditions form neighborhoods. It is exploratory: distance in a t-SNE plot does not prove physical similarity or causality. Its value is qualitative inspection alongside the numeric metrics and error examples.

DCRNN confusion matrix

Actual / Predicted	Normal	Warning	Critical
Normal	1504	44	0
Warning	45	257	20
Critical	1	41	104

For DCRNN, most Normal cases are correctly identified: 1,504 of 1,548 Normal examples. Critical cases are correctly identified 104 times, but 41 are predicted as Warning. This is a more informative view than accuracy alone because it shows the severity of the confusion. Fusion GRU has a similar pattern but captures more Critical cases in the saved matrix.

9.6 Metric interpretation

Metric	What it answers	Project reading
RMSE	How large are errors, with large mistakes emphasized?	Fusion GRU is lowest among trained models; Persistence is close and simpler
MAE	What is the average absolute MW error?	Persistence is 375.8 MW; Fusion GRU is 400.3 MW

Metric	What it answers	Project reading
Accuracy	What fraction of stress labels are correct?	Fusion GRU 92.7%, DCRNN 92.5%, Persistence 91.4%
Precision	When the model predicts a class, how reliable is it?	Macro precision is around 85% for trained models
Recall	How many actual class cases were found?	Fusion GRU recall is 86.2%; DCRNN recall is 82.7%
F1	What is the balanced class score?	Fusion GRU has the best saved F1 at 85.6%
Inference time	How quickly can one prediction be produced?	Persistence is fastest; DCRNN is slower at 63.6 ms

9.7 Results summary

The results answer the research questions with a nuanced conclusion. A trained temporal model can improve RMSE and class metrics over Persistence, but the gain is small in RMSE and comes with additional complexity. Graph diffusion is measurable and reproducible but does not win here. Feature importance indicates that local recent history dominates. The prescriptive layer is therefore valuable mainly because it connects forecast evidence to a transparent what-if action, not because the model has proven operational control.

CHAPTER 10 PRESCRIPTIVE ANALYTICS AND REROUTING

10.1 Decision sequence

1. Select a visible interconnector and inspect its current flow, utilization, risk class, distance, and proxy loss.
2. Choose a reduction percentage p, clamped between 0% and 60%.
3. Compute the amount of flow to investigate moving.
4. Remove the selected link from the working graph and search paths between the same endpoints.
5. Reject or penalize links that lack enough proxy headroom or are Critical.
6. Rank candidates using distance, utilization, warning/critical penalties, and estimated loss.
7. Return route status, route nodes, route edges, bottlenecks, baseline loss, rerouted loss, and validation requirements.

$$\text{moved MW} = \frac{\text{current MW} \times \text{reroute percentage}}{100}$$

$$\text{headroom} = \text{proxy capacity} - \text{current flow}$$

$$\text{route cost} = \text{distance} \times (1 + 2.2\,\text{utilization} + 6\,\text{loss percentage})$$

10.2 Why this is prescriptive

Forecasting reports a likely future value. The route planner evaluates a possible intervention under constraints. It is prescriptive because it compares alternatives and explains the preferred candidate, but it remains a recommendation because it does not know verified thermal ratings, voltage stability, N-1 contingencies, protection settings, or operator priorities.

CHAPTER 11 DISCUSSION NOVELTY LIMITATIONS AND ETHICS

11.1 What is new in this project

- Canonical edge audit that exposes directional aliases instead of hiding them.
- Data-quality-aware risk screen combining utilization, imbalance, and load residual.
- Evidence-driven model router that compares simple and neural approaches under chronological validation.
- Explicit separation of measured flow, estimated capacity, and distance-based loss proxy.
- Counterfactual headroom and stress-aware route search with best-effort bottlenecks.
- Action queue that includes reasons, confidence, and validation checks.

11.2 Technical limitations

- The physical-loss calculation uses centroid distance and assumed voltage, power factor, resistivity, and area.
- The capacity proxy is derived from observed flow and is not a thermal rating.
- The historical replay is not live telemetry.
- The graph does not contain full electrical admittance, phase, voltage, or reactive-power information.
- The model comparison has one supplied network and one evaluation protocol.
- Uncertainty bands are not calibrated probability intervals.
- No automatic dispatch or switching action is permitted.

11.3 Ethical and operational safeguards

The main ethical risk is overtrust: a polished dashboard may be mistaken for an operational authority. The system counters this by labeling proxy values, naming missing parameters, showing failure cases, retaining baseline comparisons, and displaying engineering checks beside routes. A future operational version would require asset validation, secure telemetry, access control, audit logs, calibrated uncertainty, N-1 analysis, and approval by qualified grid personnel.

CHAPTER 12 CONCLUSION AND FUTURE WORK

12.1 Conclusion

GridPulse demonstrates a complete predictive and prescriptive analytics workflow for cross-border power-flow data. It begins with two raw CSV datasets, validates and canonicalizes them, builds a 12-node and 24-edge graph, extracts local, global, and graph features, constructs 24-hour windows, forecasts next-hour country net flow, evaluates continuous and class-aware performance, estimates a transparent distance-based loss proxy, and tests alternate routes under visible guardrails.

The saved results show that Fusion GRU is the strongest trained research model by RMSE and F1, while Persistence remains a competitive and simpler operational baseline. DCRNN is successfully implemented and measured but lags the other trained model on this dataset, which is an informative result rather than a failure of the project. The project contribution is an auditable system integration that helps a new user understand not only what the model predicts but also how the recommendation is formed and where engineering validation is still required.

12.2 Team contribution

Member	Explanation responsibility
Prakki Krishna Karthik	Problem, datasets, preprocessing, graph and feature engineering, physical assumptions
Vasudev	Forecasting models, evaluation equations, APIs, loss calculation, and route decision logic
Shubham Bhuyan	DCRNN, saved figures, comparative interpretation, frontend integration, novelty, and limitations

12.3 Future work

1. Add verified voltage, conductor, route geometry, thermal rating, reactive power, and N-1 security data.
2. Compare GCN, GAT, GraphSAGE, Graph WaveNet, and stronger spatio-temporal baselines under the same split.
3. Add calibrated conformal or quantile prediction intervals and evaluate coverage and width.
4. Add weather, market, outage, and renewable forecast scenarios without leakage.
5. Validate route recommendations against an engineering power-flow solver and operator constraints.
6. Replace historical replay with a secure Kafka, MQTT, or utility API adapter after governance review.

REFERENCES

[1] Supplied project dataset, dynamic_with_flows.csv, hourly country-level dynamic records, 2022–2024.

[2] Supplied project dataset, power_flow.csv, directional interconnector flow records, 2022–2024.

[3] GridPulse Software Requirements Specification, project artifact, docs/SRS.md.

[4] Y. Li, R. Yu, C. Shahabi, and Y. Liu, Diffusion Convolutional Recurrent Neural Network Data Driven Traffic Forecasting, ICLR 2018. https://arxiv.org/abs/1707.01926.

[5] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang, Graph WaveNet for Deep Spatial Temporal Graph Modeling, Proceedings of IJCAI 2019, pp. 1907–1913. https://arxiv.org/abs/1906.00121.

[6] K. Cho et al., Learning Phrase Representations using RNN Encoder Decoder for Statistical Machine Translation, EMNLP 2014, pp. 1724–1734. https://arxiv.org/abs/1406.1078.

[7] J. Lu, H. Tan, R. Zhao, W. Guo et al., Applications of Graph Computing and Graph Neural Networks in Power Systems A Survey, CICED 2024, DOI 10.1109/CICED63421.2024.10754312.

[8] A Survey on Graph Neural Networks for Time Series Forecasting Classification Imputation and Anomaly Detection, IEEE TPAMI, 2024, DOI 10.1109/TPAMI.2024.3443141.

[9] scikit-learn User Guide, Classification metrics and model evaluation. https://scikit-learn.org/stable/modules/model_evaluation.html.

[10] PyTorch Documentation, Recurrent layers and neural-network modules. https://pytorch.org/docs/stable/nn.html.

[11] GridPulse research review and novelty rationale, project artifact, docs/RESEARCH_NOTES.md.

APPENDIX A COMPLETE FEATURE INVENTORY

Name	Group	Source or construction
local net flow	local time series	net_flow
local load	local time series	Actual_Total_Load_MW
local imbalance	local time series	flow_imbalance_ratio
local load residual	local time series	load_residual
global net flow	network context	all country net_flow
global load	network context	all country load
global imbalance	network context	all country imbalance
global residual	network context	all country load residual
degree centrality	static graph	canonical graph
closeness centrality	static graph	canonical graph
betweenness centrality	static graph	canonical graph

APPENDIX B PROJECT ARTIFACT MAP

Artifact	Purpose
backend/pipeline.py	Standard-library preprocessing, canonicalization, baseline evaluation, graph and physical-loss artifacts

Artifact	Purpose
backend/experiments.py	Fusion GRU and DCRNN training, evaluation, figures, and checkpoints
backend/server.py	REST and SSE services for the dashboard
backend/physics.py	Haversine, resistance, current, and loss-proxy calculations
backend/routing.py	Alternate-path search, headroom checks, cost ranking, and bottlenecks
frontend/app.js	Dashboard state, charts, replay animation, and what-if interaction
results/figures	Saved original result figures used in this report and presentation
results/models	Saved Fusion GRU and DCRNN checkpoints

APPENDIX C DCRNN TRAINING OUTPUT

The DCRNN experiment was trained for 40 epochs using a 24-hour lookback, 12 country nodes, 11 engineered features, two diffusion steps, and a hidden size of 24. The training and validation records below are read from the saved experiment artifact rather than reconstructed for this report.

$$X \in \mathbb{R}^{B \times 24 \times 12 \times 11}$$

DCRNN input tensor: batch, lookback, nodes, and features.

$$H_t = z_t \odot H_{t-1} + (1 - z_t) \odot \tilde{H}_t$$

Graph-informed recurrent update.

Training output	Value
Epochs	40
Final training loss	0.01998
Final validation loss	0.02712
Final training accuracy	93.06%
Final validation accuracy	90.67%
Final training macro F1	87.60%
Final validation macro F1	83.99%
Test RMSE	633.96 MW
Test MAE	436.22 MW
Test inference time	63.63 ms

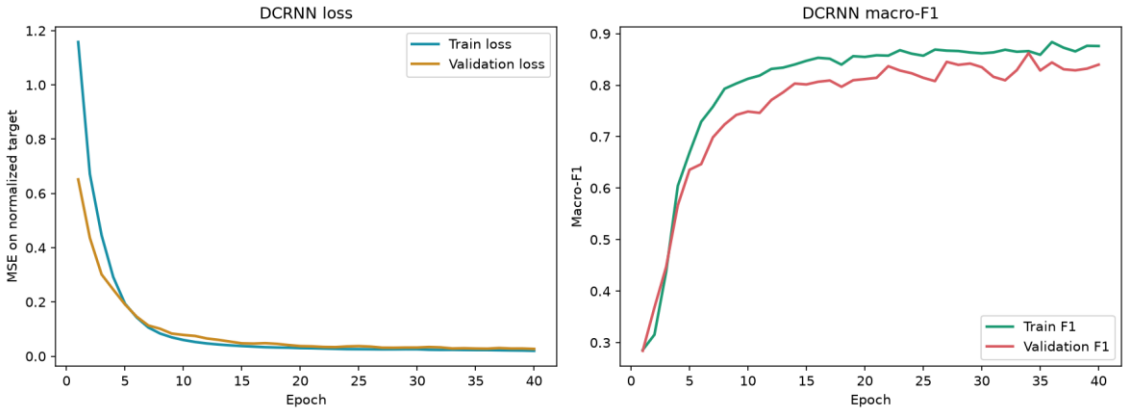


Figure 10 Original saved DCRNN training output with loss and macro F1 traces.

The final DCRNN result is useful even though it does not beat Fusion GRU on this dataset. It demonstrates a complete graph-recurrent implementation, produces a saved checkpoint at results/models/dcrnn.pt, and provides evidence about the current topology and feature contract. Its lower score is interpreted as a dataset and topology finding, not as evidence that graph learning is universally ineffective.

APPENDIX D DATASET SCHEMA AND PROVENANCE

The project separates provenance from derived analytics. The two supplied CSV files remain the source observations. The processed JSON files are deterministic contracts generated by the pipeline and are used by the API, dashboard, and experiment runner. This separation makes it possible to reproduce a result and identify whether a number was measured, derived, or assumed.

Dataset element	Meaning	Used by
Country record	One country observation at one UTC hour	Node snapshot, feature tensor, forecast target
Directional flow record	One source-to-target interconnector observation	Edge direction, magnitude, utilization
Canonical edge	Sorted endpoint identity after alias normalization	Graph topology and route search
Processed series	Aligned arrays for load, net flow, imbalance, residual, and timestamps	Server replay and model windows
Manifest	Counts, quality flags, assumptions, centrality, and graph metadata	Validation and frontend data contract

Provenance rules

- Raw row counts are not changed silently; preprocessing records duplicate, missing, and gap counts.
- Country and interconnector aliases are retained as audit metadata while the model uses canonical labels.
- The replay cursor selects an existing timestamp; the stream does not generate random synthetic observations.
- Figures are exported from saved JSON or CSV result artifacts and are not redrawn inside the report.
- The physical-loss result is explicitly labelled a proxy because the source data lacks surveyed conductor parameters.

This provenance design is important for a predictive analytics course project because the same model score can be misleading if the data split, target definition, or preprocessing operation changes. GridPulse therefore stores the configuration and data-quality evidence beside the model outputs.

APPENDIX E COMPLETE FEATURE ENGINEERING DETAILS

The feature tensor is deliberately compact. Each country receives four local signals, four system-wide summary signals, and three static graph descriptors. The construction gives Fusion GRU enough context to model temporal behaviour while allowing DCRNN to preserve the country dimension and exchange information through transitions.

Feature	Construction	Reason for inclusion
Local net flow	Observed country net flow at t	Autoregressive anchor for the target
Local load	Observed country load at t	Demand pressure changes transfer requirement
Local imbalance	Inflow/outflow imbalance ratio	Separates stable transfer from directional stress
Local load residual	Actual load minus day-ahead forecast	Captures unexpected demand error
Global net flow	Mean net flow across countries	Represents system-wide transfer state
Global load	Mean load across countries	Represents shared demand conditions
Global imbalance	Mean absolute imbalance	Measures collective directional pressure
Global residual	Mean absolute load residual	Signals a difficult operating period
Degree centrality	degree(node)/(N−1)	Counts local connectivity
Closeness centrality	Inverse mean shortest-path distance	Measures graph proximity
Betweenness centrality	Fraction of shortest paths through node	Identifies bridge nodes

$$x' = (x - _train) \,/\, \sigma_train$$

Training-only standardization; validation and test data never determine the scaling statistics.

The target is next-hour country net flow. A 24-hour window is used so the model can observe one complete daily cycle while remaining small enough for the supplied data. Samples are generated separately for training, validation, and the final chronological test horizon. This avoids leakage from later hours into earlier training windows.

APPENDIX F DCRNN COMPUTATION WALKTHROUGH

The DCRNN implementation in backend/experiments.py is compact enough to audit line by line. It first builds forward and reverse row-normalized transition matrices from the canonical graph. Self-loops are added so that a node retains its own signal while receiving information from connected nodes.

$$X^{(0)} = X, \quad X^{(k)} = PX^{(k-1)}$$

Repeated transition multiplication propagates a signal k graph steps.

$$H = [X^{(0)} \| X_f^{(1)} \| X_f^{(2)} \| X_r^{(1)} \| X_r^{(2)}]W + b$$

The local, forward, and reverse supports are concatenated before the graph convolution output.

1. Receive the 24-hour window containing all 12 countries and all 11 features.
2. At each hour, concatenate the current node input with the previous hidden state.
3. Apply diffusion convolution to create update and reset gate values.
4. Use the reset gate to form a candidate hidden state from current input and prior memory.
5. Combine old memory and candidate memory using the update gate.
6. After the final hour, apply LayerNorm and a linear head to predict one flow value per country.

The important difference from Fusion GRU is not that DCRNN has a recurrent unit; both models have temporal memory. The difference is that DCRNN performs explicit graph propagation before the recurrent update, whereas Fusion GRU receives graph information through engineered global summaries and centrality features. This makes DCRNN a direct test of the graph hypothesis.

APPENDIX G BACKEND API AND LIVE REPLAY DATA FLOW

The backend is intentionally lightweight. server.py loads the processed artifacts at startup and exposes JSON endpoints. The frontend requests static evidence once, then receives replay snapshots through Server-Sent Events. The backend does not require a database for the course demonstration because the processed series and result files are versioned project artifacts.

Endpoint	Input	Calculation or response
/api/metadata	None	Manifest, quality, graph, model and assumption metadata
/api/snapshot	Timestamp index	Nodes, edges, risk classes, loss proxy, graph analysis
/api/forecast	Country, model, horizon	History plus deterministic forecast and adaptive band
/api/metrics	None	Baseline metrics and offline Fusion GRU/DCRNN evidence
/api/stream	SSE connection	Next historical snapshot approximately every two seconds
/api/what-if	Edge ID and reduction percent	Projected flow, route plan, bottlenecks and loss delta

Snapshot calculation

For a selected replay index, server.py reads country arrays and edge arrays at the same timestamp. It derives utilization, direction, loss, node score, and class labels, then calls graph_analysis.py for propagated influence and edge criticality. The complete snapshot is serialized as JSON and rendered by frontend/app.js.

Interface honesty

The browser now surfaces Fusion GRU as the research model while labelling Persistence as the operational baseline. The moving particles are an animation of historical observed transfer. This distinction is important because the advanced models are trained research artifacts, while the current replay endpoint is designed for speed and transparency.

APPENDIX H WORKED REROUTING CALCULATION

The what-if planner is a recommendation engine, not an automatic control loop. The example below shows how a selected link is evaluated. Suppose the CH–DE link carries 1,414.75 MW and the operator tests a 25% reduction. The amount to investigate moving is calculated before path search.

$$\text{moved MW} = \frac{\text{current MW} \times \text{reroute percentage}}{100}$$

Scenario item	Value	Interpretation
Selected edge	CH–DE	Edge temporarily removed from the working graph
Current magnitude	1,414.75 MW	Observed replay value
Reduction	25%	User-selected scenario share
Moved transfer	353.69 MW	Flow investigated on alternate path
Candidate path	CH → AT → DE	Connected best-effort route
Loss change	+37.00 MW	Higher proxy loss; requires review

Path ranking

The route search uses a Dijkstra-style traversal. Strict search rejects an edge when its proxy headroom is less than the moved transfer or when its screening class is Critical. Surviving edges receive a cost based on distance, utilization, and loss. Warning and Critical links receive additional penalties. If no strict route exists, the planner returns a best-effort route with bottlenecks so that infeasibility is visible rather than hidden.

$$\text{route cost} = \text{distance} \times (1 + 2.2\,\text{utilization} + 6\,\text{loss percentage})$$

APPENDIX I EVALUATION METRIC INTERPRETATION

The project reports both regression and classification metrics because next-hour flow is numeric while operational screening is categorical. A model can have a strong average error and still miss Critical conditions. The report therefore combines continuous error, class balance, threshold curves, and confusion matrices.

Metric	Formula or meaning	Interpretation in this project
MAE	Mean absolute MW error	Persistence 375.8 MW; Fusion GRU 400.3 MW
RMSE	Square root of mean squared error	Fusion GRU 570.5 MW; lower is better for large mistakes
Accuracy	Correct labels / all labels	Fusion GRU 92.7%; DCRNN 92.5%
Precision	TP/(TP+FP)	Reliability when a risk class is predicted
Recall	TP/(TP+FN)	Ability to find actual risk cases
F1	Harmonic mean of precision and recall	Fusion GRU 85.6%; balances false alarms and misses
Inference time	Time to produce predictions	DCRNN 63.6 ms; graph diffusion costs more

$$F_1 = \frac{2\,\mathrm{Precision}\,\mathrm{Recall}}{\mathrm{Precision} + \mathrm{Recall}}$$

Why thresholds are based on the training data

The classification experiment uses absolute net-flow magnitudes at the 75th and 90th training quantiles for Warning and Critical. This makes the boundary data-derived rather than arbitrary. The dashboard node screen also uses a 0–100 composite score with Normal below 65, Warning from 65 to 84, and Critical at 85 or above. These are two different layers: the first labels model targets; the second summarizes live graph-screening pressure.

APPENDIX J TESTING AND VALIDATION MATRIX

Testing covers both numerical behavior and service integration. Unit tests do not prove engineering validity, but they reduce the chance that a route, loss, threshold, or API response is wrong because of a software defect.

Test area	Evidence	What it protects
Pipeline	tests/test_pipeline.py	Distance-loss, preprocessing, threshold and artifact behavior
Routing	tests/test_routing.py	Feasible, no-change, best-effort and bottleneck cases
API	tests/test_api.py	Endpoint availability and response structure
Layout	Rendered report pages and figure inspection	Clipping, overlap, equations and page geometry
Model evaluation	Chronological holdout and saved JSON metrics	Leakage control and reproducibility
Data quality	Manifest missing, duplicate and gap counts	Visibility of incomplete observations

Validation boundaries

- The capacity proxy is not a verified thermal rating.
- The loss estimate is not a surveyed line-loss study.
- The graph topology is not a complete electrical admittance matrix.
- The recommendation is not an N-1 security assessment.
- The historical stream is not connected to live utility telemetry.

These boundaries are part of the validation result. A technically responsible analytics system must describe what its evidence supports and what remains outside the experiment.

APPENDIX K REPRODUCIBILITY AND ARTIFACT MAP

A new user can reproduce the dashboard and research artifacts using the following sequence from the project root. The pipeline step is needed when the source data or preprocessing code changes; the server step is sufficient for replaying existing processed artifacts.

Command	Purpose
python backend/pipeline.py	Read CSVs, validate, canonicalize, build processed JSON and baseline metrics
python backend/experiments.py	Train Fusion GRU and DCRNN, save checkpoints, metrics and figures
python backend/server.py	Start the local REST and SSE dashboard service
Open http://localhost:8000	Use the browser interface for replay, forecast and what-if routing

Saved evidence

- results/model_comparison.json stores the common-holdout metrics.
- results/dcrnn_training_curves.json stores epoch-level training output.
- results/models/fusion_gru.pt and results/models/dcrnn.pt store trained checkpoints.
- results/figures contains the original exported charts used in the report and presentation.
- data/processed/manifest.json records graph structure, quality flags, and physical assumptions.

The final implementation therefore has two reproducible modes. The research mode trains and evaluates neural models with PyTorch. The dashboard mode loads compact artifacts and runs a lightweight local replay service. The report keeps the modes distinct so a reader can understand both the research result and the practical interface behavior.